

Player-AD — Manual de Arquitetura para Desenvolvimento

Documento técnico para onboarding e manutenção do `Player-AD`.

1. Visão geral

O `Player-AD` é um player Android TV orientado a:

- buscar plano de mídia no backend (`heartbeat` + `dispatch`);
- fazer cache local de mídias;
- reproduzir vídeos/imagens em loop;
- operar em modo offline com fallback local;
- expor tela de debug para configuração e diagnóstico.

2. Blocos estruturais

2.1 Bootstrapping e ciclo de vida

- `PlayerAdApplication`
 - inicializa logger (`PlayerAdLogger.init`);
 - semeia fallback local a partir de assets (`FallbackSeeder.seedFromAssetsIfNeeded`);
 - cria e inicializa `MediaCacheManager`.
- `BootCompletedReceiver`
 - recebe `ACTION\_BOOT\_COMPLETED`;
 - inicia `MainActivity` automaticamente.

2.2 UI e Orquestração

- `MainActivity`
 - tela principal em modo kiosk/imersivo;
 - cria `ExoPlayer` + `PlayerView` + `ImageView`;
 - inicia `PlayerController.start()`;
 - abre `DebugConfigActivity` no primeiro arranque ou por gesto oculto (8 taps/clicks).
- `DebugConfigActivity`
 - edição de `serverUrl`, `uin`, `deviceId`;
 - testes de conectividade (`heartbeat`, `dispatch`);
 - exibe log operacional e estado offline;
 - persiste `player-config.json` interno (e tenta externo).

2.3 Integração backend

- ``DispatcherApiClient``
 - ``getToken()``, ``heartbeat()``, ``getDispatchPlan(token)``;
 - retry controlado em ``401`` com renovação de token;
 - ``resolveUrl()`` para transformar URL relativa em absoluta.
- ``PlayerEventsClient``
 - envia telemetria de playback em ``/api/player/event``;
 - recupera token automaticamente (``heartbeat`` / ``getToken``) quando recebe ``401``.

2.4 Playback e fallback

- ``PlayerController``
 - parseia ``DispatchPlan``;
 - faz pré-cache assíncrono;
 - executa loop de playback;
 - persiste último plano online e fonte do plano;
 - fallback na ordem:
 1. plano online;
 2. plano persistido em disco;
 3. fallback local ``propagandas/vinhetas``.

2.5 Armazenamento local

- ``AppDirs``
 - resolve raiz de dados (``externalFilesDir`` com fallback para ``filesDir``);
 - padroniza diretórios:
 - ``propagandas/``
 - ``vinhetas/``
- ``MediaCacheManager``
 - mantém ``metadata.json``;
 - controla validade de arquivo em cache;
 - aplica limpeza LRU (janela > 8 dias).
- Persistência operacional
 - log: ``filesDir/player-ad-operations.log``;
 - último plano: ``<AppDirs.root>/last-dispatch-plan.json``;
 - fonte atual do plano: ``<AppDirs.root>/current-plan-source.txt``.

3. Fluxo principal de execução

1. App inicia (`PlayerAdApplication.onCreate`).
1. `MainActivity.onCreate`:
 - configura UI;
 - cria player;
 - decide entre abrir debug (first-run) ou iniciar player.
1. `PlayerController.start`:
 - `heartbeat`;
 - `dispatch`;
 - persiste plano;
 - pré-carrega mídias;
 - entra em loop.
1. A cada ciclo completo da playlist:
 - tenta novo `heartbeat + dispatch`;
 - atualiza plano e cache quando disponível.
1. Em falha de rede/dispatch:
 - tenta plano persistido;
 - senão, fallback local.

4. Classes e métodos principais

`MainActivity`

- `startPlayer`
 - carrega config via `PlayerConfigLoader`;
 - instancia `DispatcherApiClient` e `PlayerController`;
 - dispara coroutine para `playerController.start`.
- `openDebug(reason)`
 - interrompe playback em execução;
 - abre `DebugConfigActivity` com motivo.
- `registerDevTap`
 - contador temporal para desbloquear debug por interação oculta.
- `enterImmersiveMode`
 - aplica modo fullscreen/kiosk (WindowInsets + fallback legado).

`PlayerController`

- `start`

- handshake inicial online + fallback inteligente.
 - ``parseDispatchPlan(json)``
- converte JSON para ``DispatchPlan`` tipado.
 - ``preloadPlan(plan)``
- limpa cache e baixa mídias ausentes.
 - ``playLoop(plan, token, initialSource)``
- loop principal de reprodução com atualização periódica do plano.
 - ``playItem(plan, item, token)``
- trata imagem/vídeo;
- envia eventos de início/fim;
- atualiza token retornado pelo ``PlayerEventsClient``.
 - ``buildFallbackPlan()``
- gera playlist local N:1 (``propagandas:vinheta``) via ``fallbackPropagandasPerVinheta``.
 - ``saveDispatchPlanToDisk()`` / ``loadDispatchPlanFromDisk()``
- persistência do último dispatch válido.
 - ``updatePlanSource()``
- registra e persiste origem atual do plano (``ONLINE``, ``PERSISTED``, ``FALLBACK_LOCAL``).

``DispatcherApiClient``

- ``getToken()``
- busca token em ``/api/player/token``.
 - ``heartbeat()``
- mantém sessão viva e atualiza token.
 - ``getDispatchPlan(token)``
- obtém plano em ``/api/player/dispatch``.
 - ``resolveUrl(relativeOrAbsolute)``
- normaliza URLs para download/stream.

``PlayerEventsClient``

- ``sendEvent(...)``
- envia eventos (``video_playback_start/end``, ``image_display``, etc.);
- em ``401``, tenta renovação de token e reenvio.

``MediaCacheManager``

- ``init()``

- carrega ``metadata.json``.
 - ``onDownloadCompleted(...)``
- grava metadados de mídia baixada.
 - ``onPlayFromCache(mediaId, today)``
- atualiza ``lastAccessed`` e contador diário ``sum_play``.
 - ``cleanupIfNeeded()``
- remove LRU de itens antigos até atingir limite.
 - ``getMetadata(mediaId)``
- retorna estado tipado do cache.

``DebugConfigActivity``

- ``runTestHeartbeat()``
- valida conectividade básica e token.
 - ``runTestDispatchPlan()``
- valida retorno do plano e apresenta resumo.
 - ``saveConfigInternal(cfg)``
- persiste config em arquivo interno.
 - ``buildOfflineStateText()``
- mostra diagnóstico de plano persistido, fonte e contagem de fallback local.

5. Contratos de dados principais

5.1 Configuração (``player-config.json``)

Campos:

- ``serverUrl`` (obrigatório)
- ``uin`` (obrigatório)
- ``deviceId`` (obrigatório)
- ``acceptImagesInPlaylist`` (opcional, default ``true``)
- ``fallbackPropagandasPerVinheta`` (opcional, mínimo ``1``, default ``3``)

Ordem de leitura (``PlayerConfigLoader``):

1. ``filesDir/player-config.json``
1. ``/sdcard/smartsignage/player-config.json``
1. defaults embutidos.

5.2 Dispatch plan

Fonte: endpoint `/api/player/dispatch`.

Campos usados:

- `plan.playlistId`
- `plan.playlistName`
- `plan.campaignId`
- `plan.mediaItems[]` com:
 - `mediaId`
 - `url`
 - `duration`
 - `mediaType`

5.3 Eventos de telemetria

Endpoint: `/api/player/event`.

Estrutura inclui:

- `eventType`
- `mediaId` / `playlistId` / `campaignId`
- `duration`
- `completed`
- `metadata` (inclui `deviceId` e `player=Player-AD`).

6. Decisões técnicas relevantes

- **Fallback resiliente:** player não depende 100% da rede para iniciar/continuar.
- **Token resiliente:** backend `401` não interrompe playback; tenta renovação.
- **Imagens fora do ExoPlayer:** exibidas via `ImageView` para reduzir complexidade.
- **Persistência de estado operacional:** logs e fonte de plano ficam acessíveis no debug.
- **Compatibilidade Android TV/STB:** `AppDirs` evita dependência rígida de storage externo.

7. Pontos de atenção para evolução

- `MainActivity.currentVersionCode()` usa API legada (`versionCode` depreciado).
- `PlayerEventsClient.sendEvent()` ignora falhas de telemetria por design (não derruba playback).
- Limpeza LRU considera apenas mídias válidas sem acesso recente (> 8 dias); revisar estratégia para cenários de storage crítico.
- `DebugConfigActivity` hoje preserva `fallbackPropagandasPerVinheta` carregando valor atual; se virar campo editável na UI, ajustar `readConfigOrNull`.

8. Checklist para mudanças seguras

- Alterou contrato backend?
- revisar `DispatcherApiClient`, `PlayerEventsClient`, `parseDispatchPlan`.
- Alterou mídia/layout de playback?
- testar vídeo + imagem + fallback local.
- Alterou persistência?
- validar `last-dispatch-plan.json`, `current-plan-source.txt`, `metadata.json`.
- Alterou fluxo de boot/kiosk?
- validar `BootCompletedReceiver`, first-run debug, retorno de `DebugConfigActivity`.
- Alterou instalação/distribuição?
- atualizar scripts em `install-pendrive/scripts` e documentação associada.

9. Referências internas

- `Player-AD/docs/REGISTRO-OPERACIONAL.md`
- `Player-AD/docs/TROUBLESHOOTING-CRASH.md`
- `install-pendrive/README.md`